

TICKETING MANAGEMENT SYSTEM

System Documentation — V 5.0.0.0

Organization: Akshay Software Technologies Private Limited.

Document Status Approved | **Effective Date:** July 2026

Table of Contents

- 1. Introduction & System Overview.
- 2. Project History & Background.
- 3. Project Ownership & Initiation.
- 4. System Objectives & Vision.
- 5. Key Features & Functional Modules.
- 6. Technical Architecture & Stack.
- 7. Development & Implementation Details.
- 8. Time & Resource Investment.
- 9. System Benefits & Business Impact.
- 10. Security, Compliance & Reliability.
- 11. Plan of Action & Testing Approach.
- 12. Development History & Version Timeline.
- 13. Management Review Summary.
- 14. Next 12-Month Roadmap (Management View).
- 15. One-Page Executive Summary.
- 16. Formal Sign-Off & Approval Page.
- 17. Risk Register & Mitigation Plan.
- 18. Deployment & Rollback Strategy.
- 19. Appendix — Glossary, Abbreviations & References.
- 20. File Structure Overview.
- 21. Conclusion & Management Summary

1. Introduction & System Overview

1.1 Overview

The **Ticketing Management System (TMS)** is a centralized, role-based digital platform designed to manage, track, and resolve operational, technical, and service-related requests across the organization. The system provides a structured, transparent, and auditable workflow for handling tickets raised by internal users or customers and processed by support and administrative teams.

The platform is designed with **scalability**, **security**, and **operational efficiency** in mind, ensuring it can support both current organizational needs and future enterprise growth.

1.2 Business Problems Addressed

The system was developed to address several critical business challenges:

| Problem | Impact Before TMS |

- 1. Lack of centralized request and issue tracking.
- 2. Tickets lost across emails and chat channels
- 3. Manual or email-based ticket handling.
- 4. Delays, miscommunication, and missed follow-ups
- 5. No standardized prioritization or SLA enforcement.
- 6. Critical issues treated equally to minor requests.
- 7. Limited management visibility into workload and performance.
- 8. No real-time data for informed decisions.
- 9. Inconsistent accountability and escalation processes.
- 10. Unclear ownership across departments
- 11. Difficulty in auditing historical issues and actions.
- 12. Compliance gaps during reviews

1.3 Target Users

The Ticketing Management System is built for:

User Type	Description
Client Users	Create, submit, and track service requests, monitor ticket progress, communicate with support teams, and review ticket resolution history.
Consultants	Manage assigned tickets, provide resolutions, record work hours, update ticket status, collaborate with team members, and escalate issues when required.
Super Administrators	Exercise complete system governance, including organization-wide configuration, user and role management, security settings, analytics, reporting, system administration, and enterprise-level oversight.

2. Project History & Background

2.1 Project Initiation

The project was initiated in response to increasing operational complexity and growing service demands within the organization. As business operations expanded, existing manual and semi-digital processes became insufficient to handle the volume and criticality of support requests.

2.2 Business Challenges

Key challenges that led to project initiation included:

- ❖ **Increasing ticket volumes** without proportional support structure
- ❖ **Delayed response times** impacting internal productivity and customer satisfaction
- ❖ **Lack of real-time visibility** into issue status and assignment accountability
- ❖ **Difficulty in enforcing accountability** across departments and consultants

2.3 Strategic Motivation

The system was conceived as a **strategic enabler** to:

- ❖ Standardize issue management processes across the entire organization
- ❖ Improve service reliability and reduce average response and resolution times
- ❖ Provide management with real-time, actionable operational insights
- ❖ Establish a foundation for future automation and AI-driven intelligent support

3. Project Ownership & Initiation

3.1 Ownership

The Ticketing Management System is owned by **Akshay Software Technologies Private Limited** as an internal enterprise platform initiative.

Role	Name
Project Initiator	Rajan Chelladurai
Lead Developer	Ruhul Amin
Testing & Validation	Rajan Chelladurai, Reshma Lokhande, Ruhul Amin

> Overall ownership resides with the organization, ensuring long-term continuity, governance, and accountability.

3.2 Stakeholders

Key stakeholders involved in the project include:

- ❖ Senior Management and Leadership
- ❖ IT and Engineering Teams
- ❖ Support and Operations Teams
- ❖ Compliance and Audit Functions

3.3 Decision-Making & Approvals

Project approvals followed a **structured governance process** involving:

1. Requirement validation by operations and support teams
2. Technical feasibility assessment by IT
3. Budgetary and timeline approvals by management
4. Phased sign-offs during development milestones

4. System Objectives & Vision

4.1 Short-Term Objectives

- ❖ Centralize all ticket-related activities across the organization
- ❖ Enable role-based access and clear accountability for each ticket
- ❖ Improve response and resolution timelines significantly
- ❖ Provide basic analytics and reporting dashboards for management oversight

4.2 Long-Term Vision

1. Build a scalable enterprise-grade service management platform.
2. Enable seamless cross-departmental integration workflows.
3. Introduce intelligent automation, AI-powered ticket routing, and predictive analytics.
4. Support multi-organization / multi-company operations with centralized management.
5. Integrate AI across the platform to improve decision-making, workflow automation, and user productivity.
6. Implement AI-based conversation understanding to analyze user requests, identify intent, extract key information, and automatically categorize tickets.
7. Provide AI-powered solution recommendations by suggesting relevant knowledge base articles, previous resolutions, SOPs, and troubleshooting steps based on ticket context.
8. Enable automatic work time calculation by tracking ticket lifecycle, consultant activity, response time, and resolution time to generate accurate effort reports.
9. Introduce AI-assisted performance analytics to measure consultant efficiency, workload distribution, SLA compliance, and productivity trends.

- 10. Implement AI-driven priority prediction to automatically identify critical incidents and recommend escalation based on business impact.
- 11. Provide intelligent resource allocation by recommending the most suitable consultant based on expertise, availability, and historical performance.
- 12. Develop predictive maintenance and issue forecasting by identifying recurring problems and recommending preventive actions before incidents occur.
- 13. Integrate with ERP, CRM, HRMS, email, WhatsApp, Microsoft Teams, and other enterprise applications for end-to-end business process automation.
- 14. Support self-learning AI models that continuously improve recommendations based on historical ticket data, user feedback, and organizational knowledge.

4.3 Alignment with Business Growth

The system architecture and design align with the company's long-term growth strategy by supporting:

- Modular expansion**— new feature modules without architectural overhaul
- High availability** — designed for minimal downtime in production
- Future integrations** — built with APIs connecting to enterprise systems (CRM, ERP, ITSM)

5. Key Features & Functional Modules

5.1 Customer Login & Incident Management

- ❖ Customers securely log in to create and manage their **SAP Incidents**
- ❖ Incidents can also be raised for **CREST ERP** systems
- ❖ Upon successful ticket creation, an **automated email notification** is dispatched to the customer containing the unique Ticket ID
- ❖ CC email addresses (Division Heads, managers) are automatically included in relevant notifications

5.2 Incident Classification

Customers define the nature of the incident at the time of creation:

Incident Type	Description
Functional / Transactional	Business process or transactional flow issues
Technical / Connection	System connectivity, database, or infrastructure issues
Add-Ons	Custom development, SAP add-on or third-party integration issues

This categorization ensures accurate routing, prioritization, SLA tracking, and reporting.

5.3 Contract Status Validation

- ❖ Customers can view whether their support contract is **Active** or **Suspended**
- ❖ If the contract is suspended, users receive an **Account Suspension** message during incident creation
- ❖ Authorized ASTPL Admin users can **reactivate suspended accounts** from the management console

5.4 ASTPL Internal — Admin & Consultant Features

Feature	Description
Consultant Notification	Assigned consultants receive email alerts when a customer raises a ticket
Division Head CC	Division Heads are automatically CC'd in all ticket communications
Work Hour Logging	Consultants log time taken to resolve each ticket
Non-Ticket Time Tracking	Dedicated options for Add-On development or internal activities

5.5 Reporting & Dashboards

Customer Dashboards

- ❖ Tickets raised to date
- ❖ Open (active) tickets
- ❖ Closed tickets

Super Admin Dashboards

Dashboard Panel	Drill-Down Capabilities
Open Tickets	Customer-wise open tickets, Consultant-wise open tickets
Closed Tickets	Customer-wise closed tickets
Overdue Tickets	Customer-wise, Type-wise, Consultant-wise overdue tickets

Resolution Timeline Analysis

- ❖ Consultant-wise and customer-wise time taken to resolve tickets
- ❖ Excel export available for all reports with date-range filters
- ❖ ERP Incident Type-based categorization breakdowns

5.6 Pre-Assignment Routing Rules Engine

A dynamic rule-based assignment engine automatically routes tickets to the correct consultant upon creation:

Routing Criteria	Description
Specific Client User	Match based on the individual who raised the ticket
Route all tickets from a particular company	Route all tickets from a particular company
Specific Department	Match by department, with optional category sub-matching
ERP Incident Type	Match against multiple ERP incident types (multiselect)

- ❖ Rules are evaluated sequentially by **Evaluation Order** (lowest number = highest priority)
- ❖ The first matching rule assigns the ticket; remaining rules are skipped
- ❖ If no rules match, ticket is routed to the Super Admin

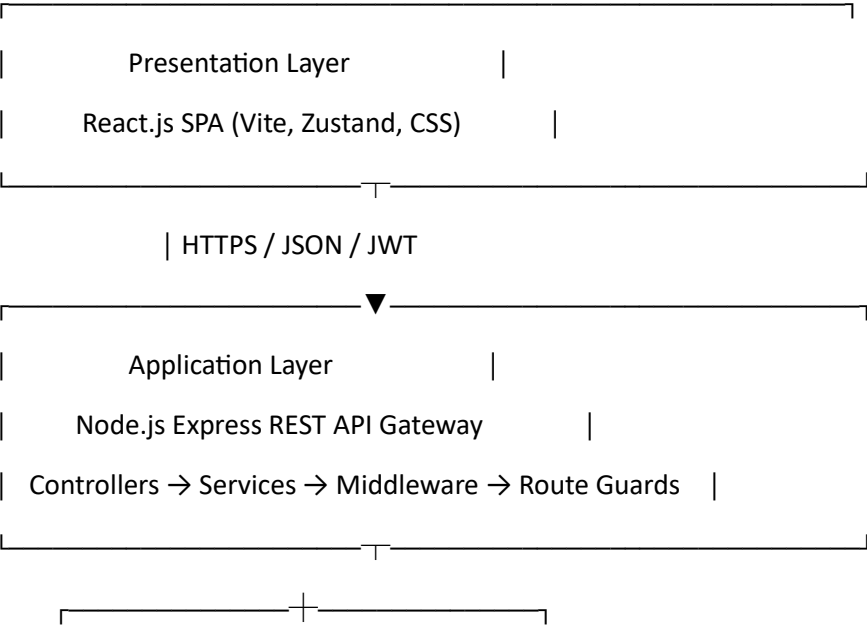
5.7 Theme Customization Engine

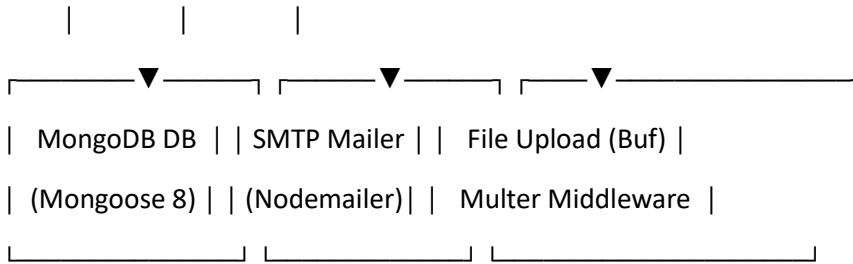
- ❖ **Theme Modes:** Light, Dark, and System (auto-detects OS preference)
- ❖ **Primary Brand Gradient:** Customizable header and primary component colors
- ❖ **Accent Color:** Configurable interactive highlights
- ❖ Live preview before applying — changes can be discarded without saving
- ❖ Preferences persist across sessions via user profile database storage

6. Technical Architecture & Stack

The Ticketing Management System is designed using a modern, modular, and scalable architecture ensuring performance, security, and ease of maintenance. The architecture follows clear separation of concerns between presentation, application logic, and data layers.

6.1 Overall Architecture





6.2 Frontend Architecture & Technologies

Category	Technology / Feature	Details
Frontend Framework	React	React 18
Build Tool	Vite	Vite 8
State Management	Zustand	Global stores per domain entity
Styling	CSS & Tailwind CSS	Vanilla CSS with Tailwind CSS utility-first approach
Theme Engine	CSS Custom Variables	Dynamic theme injection using CSS variables at the :root level
Routing	React Router	React Router v6
HTTP Client	Axios	Axios with interceptors for JWT token attachment and request handling
Animations	Framer Motion	Smooth page transitions and micro-interactions
Icons	Lucide React	Modern, lightweight SVG icon library
Architecture	Component-Based	Modular, reusable, and scalable UI architecture

Role-Specific Dashboards Rendered:

- ❖ Client User Dashboard (ticket raise, tracking, feedback)
- ❖ Consultant Dashboard (queue management, work logs, performance)
- ❖ Admin Dashboard (user management, overview reporting)
- ❖ Super Admin Console (full system control, routing rules, CC configuration)

6.3 Backend Architecture & APIs

Category	Technology / Feature	Details
Runtime Environment	Node.js	Node.js v20+
Backend Framework	Express.js	Express.js 4
Architecture	RESTful API	MVC (Model–View–Controller) based architecture with modular API design
Database ODM	Mongoose	Mongoose 8 for MongoDB schema modeling and data validation
Authentication	JWT	JSON Web Token (JWT) based authentication and authorization
File Handling	Multer	Buffer-based file uploads with documents stored in MongoDB as Binary (GridFS/Binary Storage)
Email Service	Nodemailer	SMTP transport for sending notifications, OTPs, and system emails
Password Security	bcryptjs	Password hashing using bcryptjs with a work factor (salt rounds) of 10

Layered Design:

routes/ → URL definitions and method bindings

controllers/ → Request/response handlers

services/ → Core business logic and database queries

middleware/ → JWT guard, role check, file parser

models/ → Mongoose schemas and virtuals

6.4 Database Design & Data Flow

Category	Technology / Feature	Details
Database	MongoDB	MongoDB 6+ document-oriented NoSQL database
Database ODM	Mongoose	Mongoose 8 for schema definition, validation, and database operations
Data Model	Document-Oriented	Flexible document-based schema with ObjectId references for relationships between collections

Key Collections:

Collection	Purpose	Description
clientusers	User Management	Stores all system users, including Clients, Consultants, Admins, and Super Admins, along with authentication and profile information.
clients	Client Management	Maintains client company profiles, ERP integration details, contact information, and organizational settings.
departments	Department Management	Stores organizational departments, service categories, and ticket assignment configurations.
tickets	Ticket Management	Contains complete support ticket information, including lifecycle, status, priority, SLA, assignments, attachments, and communication history.
preassignmentrules	Routing & Automation	Defines automatic ticket routing rules based on client, department, category, priority, ERP incident type, and other business criteria.
ccemailconfigs	Email Configuration	Stores CC email recipient mappings and notification rules for automated email communications.
notifications	Notification Management	Maintains in-app notification records, alerts, reminders, and user-specific system messages.
timeentries	Time Tracking	Records consultant work logs, effort hours, start/end times, billable hours, and ticket-wise time entries for reporting and performance analysis.

Data Flow:

1. Frontend (validated request) → 2. JWT Guard → 3. Role Check
→ 4. Controller → 5. Service (business logic) → 6. Mongoose Query
→ 7. MongoDB → 8. Response serialized → 9. Frontend rendered

6.5 Authentication, Authorization & Security

- ❖ **JWT-based authentication** — tokens issued on login, validated on every protected request
- ❖ **Role-Based Access Control (RBAC)** — `restrictTo(...roles)` middleware guard on every API route
- ❖ **Role Hierarchy:** `'clientuser'` → `'consultant'` → `'admin'` → `'superadmin'`
- ❖ **bcryptjs password hashing** — plaintext passwords never stored
- ❖ **Input sanitization** — Mongoose ObjectId parsing prevents NoSQL injection
- ❖ **Payload size limits** — 20MB maximum request size to prevent DoS

6.6 Email & Notification Architecture

- ❖ **Automatic email triggers** for: ticket creation, assignment, status change, resolution, closure
- ❖ **CC Distribution:** Division Heads and configured CC email addresses are automatically included
- ❖ **CC Email Config Panel:** Super Admins configure CC recipients and notification event toggles without code changes
- ❖ **Nodemailer SMTP transport** — compatible with corporate SMTP gateways
- ❖ **In-App Notifications:** Zustand-managed notification store updates on activity feed events

6.7 Hosting, Deployment & Environment Strategy

Environment	Purpose	Deployment Details
Development	Local Development	Runs on developer machines using the Vite development server with Hot Module Replacement (HMR) for rapid development and debugging.
Testing / Staging	Quality Assurance & UAT	Pre-production environment used for feature validation, integration testing, user acceptance testing (UAT), and performance verification before release.
Production	Live Deployment	Frontend is served via Nginx , while the backend is deployed on a Hostinger VPS running Node.js and managed using PM2 for high availability and automatic recovery.

Deployment Configuration

Component	Configuration
Frontend Deployment	Production build generated using npm run build, with the dist/ folder served by Nginx.
Backend Deployment	Node.js application hosted on a Hostinger VPS, managed by PM2 for process monitoring, automatic restarts, log management, and zero-downtime operation.
Web Server / Reverse Proxy	Nginx acts as the reverse proxy, routing incoming requests to the backend services, serving static assets, and supporting HTTPS/SSL.
Hosting Platform	Hostinger VPS provides a dedicated Linux server environment for hosting the backend APIs, database connectivity, and application services.
Environment Variables	Application configuration is managed through .env files. Sensitive information such as database credentials, JWT secrets, SMTP settings, and API keys is excluded from version control using .gitignore.
Process Management	PM2 ensures application resilience by automatically restarting failed processes, managing logs, monitoring performance, and enabling startup on server reboot.

Build & Release	Frontend and backend are deployed independently, allowing seamless updates, easier maintenance, and minimal downtime during releases.
-----------------	---

6.8 Logging, Monitoring & Reliability

- ❖ **Hostinger VPS Server Logs:** Runtime exceptions, API requests, application logs, and server activities are monitored through the Hostinger VPS logging environment to support troubleshooting and performance monitoring.
- ❖ **Assignment History:** Every ticket maintains a complete assignmentHistory[] trail, recording the assigned consultant, timestamp, reassignment details, and remarks for full auditability.
- ❖ **Work Log Audit:** Every time entry is linked to both the consultant ID and ticket ID, providing a complete audit trail of work performed.
- ❖ **MongoDB Audit:** All documents automatically maintain createdAt and updatedAt timestamps using Mongoose's built-in timestamp functionality.
- ❖ **Error Responses:** The API returns structured JSON error objects with standardized HTTP status codes, ensuring consistent error handling across the application.

7. Development & Implementation Details

7.1 Development Phases

Phase	Activities
Requirement Analysis & System Design	Gathered business requirements through stakeholder discussions, defined functional and non-functional requirements, prepared wireframes, designed the system architecture, database schema, user roles, and workflow diagrams.
Database Design & Backend Development	Designed MongoDB collections, developed RESTful APIs using Node.js and Express.js, implemented authentication, authorization, business logic, email services, file management, and database integration.
Core Ticketing Module Development	Developed ticket creation, assignment, status management, priority handling, SLA tracking, consultant allocation, routing rules, attachment management, and complete ticket lifecycle workflows.
Role-Based Access Control (RBAC) Implementation	Implemented JWT-based authentication, role-based authorization, secure API access, and customized dashboards and permissions for Super Admin, Admin, Consultant, and Client users.
Frontend Development & User Experience	Built responsive React-based user interfaces, dashboards, forms, data tables, filtering, search, notifications, theme management, and reusable UI components with an optimized user experience.

Automation & Business Workflow Implementation	Developed automatic ticket routing, email notifications, escalation workflows, consultant assignment logic, work-hour tracking, and configurable business rules.
Dashboard, Analytics & Reporting	Developed interactive dashboards, KPI cards, consultant performance reports, client-wise analytics, work-hour analysis, date range filtering, and Excel export functionality.
AI Integration (Roadmap)	Planned AI-powered ticket classification, intelligent routing, conversation understanding, automatic work-time calculation, solution recommendations, predictive analytics, and smart reporting capabilities.
Testing, Optimization & Quality Assurance	Performed unit testing, integration testing, user acceptance testing (UAT), bug fixing, security validation, performance optimization, and application stabilization.
Deployment & Production Release	Deployed the frontend using Nginx and the backend on a Hostinger VPS, configured environment variables, implemented HTTPS, monitored application performance, and prepared production documentation and user guides.

7.2 Team Structure

Role	Responsibilities
Project Manager / Product Owner	Defined project vision and scope, gathered business requirements, coordinated with stakeholders, prioritized features, managed project timelines, reviewed deliverables, approved milestones, and ensured successful project execution.
Technical Lead / Solution Architect	Designed the overall system architecture, established technical standards, reviewed implementation strategies, ensured scalability, security, performance, and guided the development process.
Full Stack Developer	Developed the complete application including the React frontend, Node.js/Express backend, MongoDB database, RESTful APIs, authentication, business logic, integrations, responsive UI, deployment, and system optimization.
UI/UX Designer	Designed user interfaces, dashboards, workflows, responsive layouts, user experience improvements, design consistency, and usability enhancements.
QA / Testing Contributors	Conducted functional testing, integration testing, user acceptance testing (UAT), regression testing, identified defects, verified bug fixes, and ensured application quality before production release.

DevOps / Deployment Engineer	Managed deployment on the Hostinger VPS, configured Nginx, environment variables, SSL certificates, application monitoring, backups, and production environment maintenance.
Support & Review Stakeholders	Validated business workflows, reviewed implemented features, provided domain expertise, participated in feedback cycles, and approved functional requirements and enhancements.
Client Representatives / End Users	Participated in requirement discussions, UAT sessions, feature validation, feedback collection, and final acceptance of the system based on business needs.

7.3 Challenges & Resolutions

Challenge	Resolution Applied
Complex multi-role UI rendering	Implemented role-based access control (RBAC) using JWT authentication and Zustand global state management with role-aware routing, component guards, and dynamic navigation menus.
Dynamic theme customization	Developed a flexible theming engine using CSS Custom Variables injected dynamically at the :root level, allowing runtime theme switching without recompilation.
MongoDB aggregation lookup failures	Corrected aggregation pipelines by updating lookup references from the incorrect 'users' collection to the actual 'clientusers' collection, ensuring accurate data retrieval.
ERP Incident Type multi-selection routing	Modified the database schema to store ERP Incident Types as a [String] array and implemented Array.some() matching logic to support dynamic routing based on multiple incident types.
Performance optimization for large ticket datasets	Improved application performance by creating MongoDB indexes, implementing server-side pagination, optimizing aggregation pipelines, lazy-loading React components, and minimizing unnecessary API calls.
Dynamic CC Email configuration	Introduced a configurable CcEmailConfig model with event-based toggle flags, enabling administrators to manage email recipients without modifying source code or redeploying the application.
Automatic ticket assignment complexity	Developed configurable pre-assignment rules based on client, department, category, priority, ERP incident type, and consultant mapping to automate ticket allocation.
Secure authentication and authorization	Implemented JWT-based authentication, encrypted password storage using bcrypt, protected API endpoints, and role-based middleware to prevent unauthorized access.

File upload and document management	Integrated Multer for secure file uploads with documents stored in MongoDB, including validation for supported file types and size limits.
Reporting and analytics performance	Optimized dashboard queries using MongoDB aggregation pipelines, caching strategies where applicable, and efficient filtering to generate real-time reports with minimal latency.
Accurate work-hour tracking	Implemented consultant time-entry management linked directly to tickets, enabling precise effort calculation, productivity analysis, and future AI-based automatic time estimation.
Deployment and production stability	Deployed the backend on a Hostinger VPS with Nginx reverse proxy, environment-based configuration, automated backups, and monitoring to ensure reliable production performance.

8. Time & Resource Investment

8.1 Development Timeline

Milestone	Timeline	Description
Project Initiation & Requirement Gathering	15 December 2025	Project kickoff, stakeholder discussions, business requirement analysis, workflow design, system architecture planning, and database design.
System Design & Architecture	December 2025	Finalized the application architecture, MongoDB database schema, RESTful API design, role-based access control, and UI/UX framework.
Core Development Phase	December 2025 – April 2026	Developed the core ticket management system, user management, authentication, routing engine, notifications, reporting, dashboards, and administrative modules.
Feature Enhancements & Optimization	May 2026 – June 2026	Implemented advanced reporting, consultant performance analytics, work-hour tracking, ERP incident routing, Excel exports, email automation, and performance optimizations.
Testing, UAT & Production Stabilization	June 2026 – July 2026	Conducted functional testing, User Acceptance Testing (UAT), security validation, bug fixing, deployment optimization, and production stabilization.
Final Production Release (v5.0.0.0)	03 July 2026	Successfully released Version 5.0.0.0 , featuring a production-ready enterprise ticketing platform with role-based access control, workflow automation, analytics,

		configurable routing, reporting, and enhanced system performance.
--	--	---

8.2 Effort Distribution

Activity Area	Effort (Hours)
Requirement Analysis & Initial System Design	15 Hours
Core Development (Authentication, RBAC, Ticket Management, APIs)	30 Hours
Requirement Changes, Feature Enhancements & Continuous Updates	45 Hours
Bug Fixes, Issue Resolution & System Optimization	25 Hours
Dashboard, Reporting, Routing & Advanced Features	15 Hours
Testing, UAT & Client Feedback Implementation	12 Hours
Deployment, Documentation & Production Release	8 Hours
Total Project Effort	150 Hours

8.3 Tools & Technologies

- ❖ **Version Control:** Git (branch-based development model)
- ❖ **Project Tracking:** Agile-based task management with iterative sprints
- ❖ **Code Quality:** Structured review cycles and structured error handling patterns
- ❖ **Build System:** Vite 8 (frontend), npm scripts (backend)

9. System Benefits & Business Impact

9.1 Operational Efficiency

Before TMS	After TMS (Ticket Management System)
Support requests were managed through lengthy email chains, making tracking difficult.	Centralized, web-based ticket management portal with a single source of truth for all support requests.
Ticket assignment was performed manually by coordinators, resulting in delays and inconsistencies.	Intelligent pre-assignment routing automatically assigns tickets based on configurable business rules, departments, priorities, and ERP incident types.

No standardized SLA monitoring or escalation mechanism.	Built-in SLA tracking, overdue alerts, escalation workflows, and real-time status monitoring.
Consultant work hours were maintained separately or manually.	Integrated work-hour logging, time tracking, and effort reporting directly linked to each ticket.
Limited visibility into ticket progress and consultant workload.	Real-time dashboards providing ticket status, consultant performance, workload distribution, and client-wise analytics.
Reporting required manual data collection and spreadsheet preparation.	Automated reports with advanced filtering, KPI dashboards, and Excel export functionality.
Client communication was fragmented across emails and phone calls.	Centralized ticket conversations, activity history, notifications, and email integration for better communication.
No configurable routing or business automation.	Configurable routing rules, notification workflows, email automation, and business process automation.
Difficulty tracking historical changes and audit information.	Complete audit trail including assignment history, status changes, work logs, timestamps, and user activities.
System enhancements required significant manual effort.	Scalable architecture designed for continuous enhancements, AI integration, predictive analytics, and enterprise workflow automation.

9.2 Transparency & Control

- ❖ **Real-time dashboards** for management with drill-down capabilities
- ❖ **Clear accountability** — every ticket has a named assignee and assignment history
- ❖ **Complete audit trail** — all actions timestamped and attributed to user IDs
- ❖ **Historical data** available for audits, billing reviews, and performance assessments

9.3 Customer & Employee Value

- ❖ Improved service experience through transparent ticket status visibility
- ❖ **Automated email notifications** reduce need for follow-up calls
- ❖ Predictable response timelines improve client trust and satisfaction
- ❖ Consultants benefit from structured workloads and measurable performance metrics
- ❖

10. Security, Compliance & Reliability

Domain	Implementation
Authentication	Secure JWT (JSON Web Token)-based authentication with passwords encrypted using bcrypt hashing.
Authorization	Role-Based Access Control (RBAC) implemented through middleware to protect all secured API endpoints based on user roles and permissions.
Data Access	Role-scoped database queries ensure users can access only the data they are authorized to view or modify.
Audit Logging	Comprehensive audit trail maintained through assignment history, work logs, activity records, and automatic createdAt / updatedAt timestamps.
Backup	Automated MongoDB database backups using mongodump, with backup files securely stored for disaster recovery and business continuity.
Recovery	Well-defined database restoration process using mongorestore, enabling quick recovery from backup files when required.
Application Monitoring	Backend application hosted on a Hostinger VPS with server log monitoring, application health monitoring, and runtime error tracking to ensure system reliability and availability.
Server Security	Linux-based Hostinger VPS secured with firewall rules, HTTPS/SSL encryption, restricted server access, environment-based configuration, and regular security updates.
SMTP Security	Corporate SMTP credentials, JWT secrets, API keys, and other sensitive configurations are securely stored in .env environment variables and excluded from version control.
API Security	REST APIs include request validation, input sanitization, structured error handling, and standardized HTTP response codes to prevent unauthorized access and common security vulnerabilities.

11. Plan of Action & Testing Approach

Plan of Action — I: Ticket-Based Testing

Step	Action	Expected Outcome
------	--------	------------------

1	Create a dummy client account and user profile with appropriate role permissions.	Client account is successfully created and can access the system according to assigned permissions.
2	Log in as the client user and create a new support ticket with all required details and attachments.	Ticket is successfully generated with a unique Ticket ID and enters the Open status.
3	Verify automatic ticket routing or manually assign the ticket to the appropriate consultant.	Ticket is assigned according to configured routing rules or administrator selection.
4	Log in as the assigned consultant, review the ticket, communicate with the client if necessary, and provide a resolution.	Consultant updates the ticket with comments, resolution details, and changes the status appropriately.
5	Record the time spent resolving the issue using the work-hour logging module.	Work hours are successfully linked to the ticket and reflected in consultant performance reports.
6	Verify notifications, email alerts, assignment history, and audit logs throughout the ticket lifecycle.	All notifications, email communications, activity logs, and audit records are generated correctly.
7	Close the ticket and validate reporting dashboards, analytics, and ticket history.	Ticket is marked as Closed , reports are updated, and the complete lifecycle is available for future reference.
Estimated Testing Time	Approximately 1–2 Hours per Complete Test Cycle	Covers end-to-end functional testing, workflow validation, reporting verification, and user acceptance testing (UAT).

Plan of Action — II: Non-Ticket Task Time Tracking

For tasks not directly related to customer tickets, consultants record time against predefined task categories:

Category	Use Case	Description
Add-On Development	SAP Add-On / Custom Software Development	Time spent designing, developing, enhancing, testing, and maintaining SAP Business One add-ons, custom ERP modules, APIs, and enterprise software solutions.

Category	Use Case	Description
Client Discussions	Requirement Gathering & Customer Meetings	Time allocated for client meetings, requirement analysis, demonstrations, project reviews, feedback sessions, technical consultations, and implementation planning.
	Team Collaboration & Project Planning	Time spent in internal meetings, sprint planning, design discussions, code reviews, knowledge sharing, project coordination, and cross-functional collaboration.
Research & Development (R&D)	Innovation & Technical Research	Activities involving proof of concepts (POCs), technology evaluation, AI research, framework exploration, prototype development, performance benchmarking, and technical feasibility studies.
Software Development & Bug Fixing	Feature Development & Maintenance	Time dedicated to implementing new features, resolving bugs, optimizing application performance, refactoring code, and enhancing existing system functionality.
Testing & Quality Assurance	Validation & System Verification	Functional testing, integration testing, User Acceptance Testing (UAT), regression testing, issue verification, and quality assurance activities.
Documentation	Technical & User Documentation	Preparation and maintenance of technical documentation, API documentation, user manuals, deployment guides, release notes, and project reports.
Deployment & Support	Production Deployment & Maintenance	Application deployment, server configuration, production support, monitoring, troubleshooting, backup verification, and release management.
ASTPL Internal Work	Organizational Activities	Internal administrative work, HR activities, training sessions, process improvements, organizational meetings, and company-related tasks.
General & Other Tasks	Miscellaneous Activities	Any billable or non-billable work not covered under the above categories, including miscellaneous technical assistance, learning, and operational support.

Testing & Reporting Timeline

Activity	Schedule / Frequency	Description
Daily Functional Testing	30–60 Minutes (Daily)	Perform functional testing of newly developed features, verify bug fixes, and validate critical business workflows before deployment.
Bug Tracking & Debugging Updates	Daily	Review reported issues, investigate root causes, implement fixes, and communicate progress through daily development updates.
Regression Testing	After Every Major Feature or Bug Fix	Execute regression tests to ensure newly implemented changes do not impact existing functionality or business processes.
User Acceptance Testing (UAT)	Per Version Release Milestone	Conduct end-to-end business workflow validation with stakeholders before each production release.
Performance & Stability Testing	Before Production Deployment	Verify application performance, API response times, database efficiency, and overall system stability under expected workloads.
Release Validation	Every Production Release	Validate deployment, perform smoke testing, verify critical modules, and confirm system readiness after each production deployment.
Production Monitoring	Continuous	Monitor application health, server performance, error logs, and user-reported issues to ensure reliable production operation.

12. Development History & Version Timeline

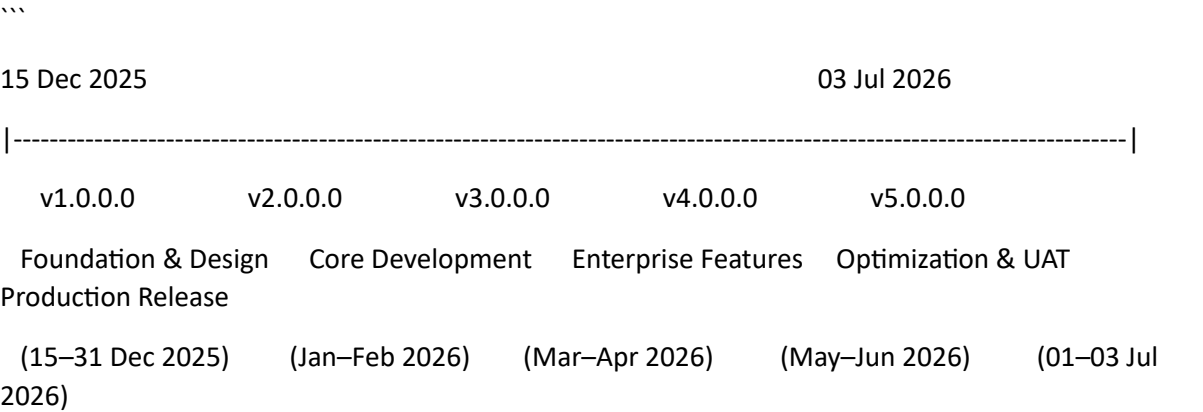
The Ticketing Management System has progressed through multiple structured versions, each representing measurable functional growth, stability improvements, and enterprise readiness.

Version History Table

Version	Phase Description	Start Date	End Date	Effort (Hours)	Key Outcomes
v1.0.0.0	Project Foundation & System Design	15 Dec 2025	31 Dec 2025	20 hrs	Requirement analysis, system architecture, database design, authentication, basic ticket creation, and project setup.

v2.0.0.0	Core System Development	01 Jan 2026	28 Feb 2026	30 hrs	User management, role-based access control (RBAC), client management, ticket lifecycle, REST APIs, notifications, and dashboard foundation.
v3.0.0.0	Enterprise Feature Expansion	01 Mar 2026	30 Apr 2026	40 hrs	Automated routing, ERP incident type mapping, work-hour tracking, email notifications, reporting modules, consultant assignment, and business workflow automation.
v4.0.0.0	Optimization & Continuous Enhancements	01 May 2026	30 Jun 2026	40 hrs	Performance optimization, dashboard improvements, analytics, Excel exports, UI/UX enhancements, bug fixes, changing requirement implementation, and production stabilization.
v5.0.0.0	Final Production Release	01 Jul 2026	03 Jul 2026	20 hrs	Final testing, UAT, deployment on Hostinger VPS, documentation, security improvements, production validation, and enterprise release.

Gantt-Style Narrative Timeline



Phase Descriptions:

Phase	Period	Focus
Phase 1 — Project Initiation & Foundation	15 Dec – 31 Dec 2025	Business requirement gathering, stakeholder discussions, system architecture, database schema design, UI/UX planning, authentication, and core ticketing workflow development.

Phase 2 — Core System Development	01 Jan – 28 Feb 2026	Development of user management, role-based access control (RBAC), client management, ticket lifecycle, RESTful APIs, notifications, dashboards, and fundamental business workflows.
Phase 3 — Enterprise Feature Implementation	01 Mar – 30 Apr 2026	Implementation of automated ticket routing, ERP incident type mapping, consultant assignment, work-hour tracking, reporting modules, email automation, audit logging, and enterprise workflow enhancements.
Phase 4 — Optimization & Continuous Enhancements	01 May – 30 Jun 2026	Performance optimization, UI/UX improvements, dashboard analytics, consultant performance reporting, Excel exports, changing requirement implementation, bug fixes, security enhancements, and production stabilization.
Phase 5 — Production Release & Documentation	01 Jul – 03 Jul 2026	Final User Acceptance Testing (UAT), production deployment on the Hostinger VPS, system validation, technical documentation, user guides, security verification, and release of Version v5.0.0.0 .

13. Management Review Summary

Delivery Overview

Item	Detail
Project Start	15 December 2025
Current Version	v5.0.0.0 (Production Release)
Project Status	Production Ready, Live & Operational with Continuous Enhancements
Delivery Model	Agile full-stack development with iterative, version-controlled releases and continuous client-driven improvements
Total Project Effort	150 Hours (90 Development + 30 Requirement Changes & Enhancements + 20 Testing, UAT & Deployment)
Development Methodology	Agile iterative development with continuous requirement analysis, feature updates, bug fixing, testing, and production optimization
Team Size	1 Full Stack Developer (Lead Developer) + QA Support & Business Stakeholders for UAT and Reviews
Deployment Environment	Frontend hosted via Nginx; Backend deployed on Hostinger VPS with MongoDB Atlas as the database

Technology Stack	React 18, Vite, Node.js, Express.js, MongoDB, Mongoose, JWT, Tailwind CSS, Zustand, Axios, Framer Motion
Future Roadmap	AI-powered ticket classification, conversation understanding, automatic work-time calculation, intelligent solution recommendations, predictive analytics, and advanced enterprise workflow automation

Quality & Governance Assessment

-  Role-based access and audit logging implemented across all system layers
-  Structured testing and daily validation cycles followed throughout development
-  Version-controlled releases (Git) with clear changelog traceability
-  Pre-assignment routing engine tested against all four condition types
-  CC email alert toggles tested end-to-end with SMTP integration

Business Readiness

-  System is **production-ready** and currently live
-  Supports contractual controls, SLA tracking, and time logging per consultant
-  Reporting dashboards provide real-time management visibility
-  Excel export capability available for filtered date range reports

Management Conclusion

> The Ticketing Management System has been delivered within a controlled timeline and effort framework, meeting both operational and governance objectives. The solution is **stable, scalable,** and **suitable for continued enterprise use** and future enhancements.

14. Next 12-Month Product Roadmap (Management View)

Quarter 1 (Months 1–3): Platform Stabilization & Operational Excellence

Focus: Enhancing platform stability, reporting, security, and user experience.

Initiative	Business Outcome
Advanced dashboard optimization and analytics	Faster reporting and improved management visibility

Initiative	Business Outcome
Enhanced reporting with scheduled Excel exports and custom filters	Accurate operational and management reporting
Consultant performance, work-hour, and workload dashboards	Better resource planning and productivity tracking
UI/UX improvements based on production feedback	Improved user adoption and operational efficiency
Security hardening, audit logging, and performance optimization	Increased reliability and enterprise readiness

Quarter 2 (Months 4–6): Enterprise Automation & System Integration

Focus: Automating business processes and integrating enterprise applications.

Initiative	Business Outcome
Intelligent ticket routing based on category, priority, workload, consultant skills, and ERP incident types	Reduced manual assignment and faster response times
ERP, CRM, HRMS, Email, and WhatsApp integration	Unified enterprise workflow and seamless data exchange
Email-to-Ticket and WhatsApp-to-Ticket automation	Simplified ticket creation across communication channels
Automated SLA monitoring, notifications, and escalation workflows	Improved SLA compliance and service quality
REST API expansion for third-party integrations	Easier integration with enterprise applications

Quarter 3 (Months 7–9): AI Integration & Intelligent Automation

Focus: Introducing AI-powered automation and decision support.

Initiative	Business Outcome
AI-powered ticket classification and intelligent routing	Automatically categorizes and assigns tickets to the most suitable consultant, reducing manual effort and improving response time
AI-based conversation understanding and intent detection	Understands customer conversations, extracts key information, detects issue intent, and automatically categorizes requests

AI-powered solution recommendations	Suggests relevant knowledge base articles, previous resolutions, SOPs, and troubleshooting steps based on ticket context
Automatic work-time calculation using AI	Estimates consultant effort based on ticket activities, conversation history, and resolution workflow, reducing manual time entry
AI-assisted consultant recommendation engine	Recommends the best consultant based on expertise, availability, historical performance, and workload
AI-powered ticket summarization	Automatically generates concise ticket summaries for consultants and management
AI chatbot for self-service support	Provides instant responses to common issues and reduces support workload

Quarter 4 (Months 10–12): Enterprise Intelligence & Predictive Analytics

Focus: Transforming the platform into an AI-driven Enterprise Service Management solution.

Initiative	Business Outcome
Predictive analytics for SLA breaches and recurring incidents	Identifies high-risk tickets before SLA violations occur and recommends preventive actions
AI-powered consultant performance analytics	Measures consultant efficiency, productivity trends, workload distribution, and SLA compliance
Management dashboards with AI-generated insights	Provides executives with intelligent business insights, trend analysis, and KPI forecasting
Predictive maintenance and recurring issue detection	Identifies recurring problems and recommends proactive resolutions before they impact operations
Multi-company and multi-tenant enterprise architecture	Supports centralized management of multiple organizations from a single platform
Self-learning AI recommendation engine	Continuously improves routing decisions, solution suggestions, and predictions using historical ticket data and user feedback
Executive AI reporting and business intelligence	Generates automated management reports, forecasts, and strategic recommendations

Roadmap Governance

Area	Approach
Release Strategy	Quarterly phased releases with continuous enhancements, maintenance updates, and hotfixes.

Development Methodology	Agile development with iterative feature delivery, sprint planning, continuous integration, and continuous improvement.
Approval Process	Quarterly roadmap reviews and milestone approvals by management and business stakeholders.
Priority Management	Features prioritized based on business value, customer feedback, operational impact, AI readiness, and strategic objectives.
Quality Assurance	Functional Testing, Integration Testing, UAT, Performance Testing, Security Validation, and AI model validation before each production release.
Continuous Improvement	Production monitoring, user feedback, analytics, AI learning models, and business metrics drive ongoing enhancements.

Strategic Vision (12-Month Outcome)

By the end of the next 12 months, the Ticket Management System will evolve into an AI-powered Enterprise Service Management (ESM) Platform capable of:

- 1. AI-powered ticket classification and intelligent routing.
- 2. AI-based conversation understanding and intent recognition.
- 3. AI-generated solution recommendations using historical resolutions and knowledge bases.
- 4. Automatic work-time calculation based on consultant activities and ticket lifecycle.
- 5. AI-assisted consultant allocation based on skills, workload, and performance.
- 6. Predictive analytics for SLA breaches, recurring issues, and operational risks.
- 7. Consultant productivity analytics and workforce planning.
- 8. Seamless ERP, CRM, HRMS, Email, WhatsApp, and Microsoft Teams integration.
- 9. Multi-company and multi-tenant enterprise support.
- 10. AI-powered executive dashboards with predictive business intelligence.
- 11. Self-learning AI models that continuously improve recommendations using historical ticket data and organizational knowledge.
- 12. Intelligent automation that minimizes manual intervention, enhances operational efficiency, and enables data-driven decision-making across the enterprise.

15. One-Page Executive Summary

Overview

The Ticket Management System (TMS) is an enterprise-grade service management platform developed and maintained by Akshay Software Technologies Private Limited to streamline, automate, and optimize the management of customer support requests, internal service operations, and enterprise workflows.

The project commenced on 15 December 2025 and has evolved through five major releases, culminating in the successful production deployment of Version v5.0.0.0 on 03 July 2026. The platform addresses key business challenges, including fragmented support processes, manual ticket assignment, limited SLA visibility, inefficient work-hour tracking, and the absence of centralized reporting and operational governance.

Designed using modern web technologies and a scalable architecture, the system provides a centralized, secure, and highly configurable platform that improves operational efficiency, enhances service delivery, and supports future enterprise growth through intelligent automation and AI integration.

Business Value Delivered

Area	Value Delivered
Incident Management	Centralized management of customer and internal support requests with complete ticket lifecycle tracking.
Workflow Automation	Automated ticket routing, configurable assignment rules, notifications, and SLA-driven workflows reduce manual effort.
Governance & Security	Role-Based Access Control (RBAC), comprehensive audit trails, and secure authentication ensure accountability and data security.
SLA & Time Management	Integrated SLA monitoring, consultant work-hour logging, overdue detection, and performance tracking improve operational efficiency.
Reporting & Analytics	Real-time dashboards, consultant performance analytics, client-wise reports, KPI monitoring, and Excel export capabilities provide actionable business insights.
Communication	Automated email notifications, configurable CC email distribution, and centralized ticket communication improve collaboration and response times.
Scalability	Modular architecture supports future ERP, CRM, HRMS, AI, and third-party integrations while enabling enterprise-scale deployment.

Delivery Snapshot

Metric	Value
Project Start	15 December 2025
Current Version	v5.0.0.0 (Production Release)
Development Effort	90 Hours
Requirement Changes & Enhancements	30 Hours
Testing, UAT & Deployment	20 Hours
Overall Project Effort	150 Hours
Development Methodology	Agile full-stack development with iterative version-controlled releases

Deployment Environment	Frontend hosted via Nginx; Backend deployed on Hostinger VPS with MongoDB Atlas
Current Status	Production Ready, Live & Operational

Strategic Outlook

The Ticket Management System is designed as a long-term enterprise platform with a clear roadmap focused on intelligent automation, digital transformation, and AI-driven service management.

Over the next 12 months, planned enhancements include:

- 1. AI-powered ticket classification and intelligent routing.
- 2. AI-based conversation understanding and intent recognition.
- 3. Automatic work-time calculation using AI.
- 4. AI-generated solution recommendations based on historical tickets and knowledge bases.
- 5. Predictive analytics for SLA breaches and recurring incidents.
- 6. Consultant productivity and workload analytics.
- 7. ERP, CRM, HRMS, Email, WhatsApp, and Microsoft Teams integration.
- 8. Multi-company and multi-tenant enterprise architecture.
- 9. Executive dashboards with AI-powered business intelligence and predictive reporting.

With its scalable architecture, configurable workflows, enterprise-grade security, and AI-first roadmap, the Ticket Management System (TMS) is positioned to become a strategic Enterprise Service Management (ESM) platform that enhances operational efficiency, supports data-driven decision-making, and enables sustainable business growth for Akshay Software Technologies Private Limited.

16. Formal Sign-Off & Approval Page

This section confirms the formal review, acknowledgment, and approval of the Ticket Management System (TMS) for production deployment, operational use, and continuous enhancement within Akshay Software Technologies Private Limited.

Ownership & Accountability

Role	Name / Organization
System Owner	Akshay Software Technologies Private Limited
Project Sponsor / Product Owner	Akshay Software Technologies Private Limited
Project Review & Business Validation	Rajan Chelladurai
Lead Developer & Technical Owner	Ruhul Amin
Testing & Quality Assurance	Rajan Chelladurai, Reshma Lokhande, Ruhul Amin

Deployment & Technical Support	Ruhul Amin
--------------------------------	------------

Approval Statement

The Ticket Management System (TMS) and the accompanying technical documentation have been reviewed and validated against the defined business, functional, technical, security, and operational requirements. Version v5.0.0.0 is approved for production deployment and enterprise use. The system is considered stable, scalable, and suitable for operational support, with future enhancements to be delivered through controlled version releases in accordance with the approved product roadmap.

17. Risk Register & Mitigation Plan

#	Risk Category	Risk Description	Impact	Likelihood	Mitigation Strategy
1	Technical	Performance degradation with increasing ticket volume and concurrent users	High	Medium	MongoDB indexing, optimized aggregation pipelines, lazy loading, pagination, caching strategies, and continuous performance monitoring.
2	Security	Unauthorized system access, privilege escalation, or data leakage	High	Low	JWT authentication, Role-Based Access Control (RBAC), bcrypt password hashing, HTTPS, input validation, audit logging, and periodic security reviews.
3	Operational	Incorrect ticket assignment, SLA breaches, or workflow failures	High	Medium	Automated routing rules, SLA monitoring, escalation workflows, validation checks, and user training.
4	Data	Data loss, accidental deletion, or database corruption	High	Low	Scheduled MongoDB Atlas backups, export snapshots, tested restoration procedures, and disaster recovery documentation.
5	Infrastructure	Hostinger VPS downtime or server resource limitations	High	Low	VPS health monitoring, regular server maintenance, backup strategy, resource monitoring, and recovery procedures.
6	Integration	Failure of SMTP, ERP, CRM, or third-party API integrations	Medium	Medium	Retry mechanisms, error logging, fallback notifications, API monitoring, and integration validation.

#	Risk Category	Risk Description	Impact	Likelihood	Mitigation Strategy
7	Requirement Changes	Frequent business requirement changes impacting delivery timelines	Medium	High	Agile development methodology, phased releases, version control, backlog prioritization, and continuous stakeholder reviews.
8	Compliance & Audit	Failure to meet organizational audit and governance requirements	Medium	Low	Complete audit trails, assignment history, activity logs, work-hour tracking, and timestamped records.
9	AI Implementation	AI-generated recommendations may require human validation during initial deployment	Medium	Medium	Human approval workflow, continuous AI model improvement, user feedback loops, and periodic model evaluation.

Risk reviews will be conducted quarterly as part of the product governance process. The Risk Register will be updated before every major version release and roadmap review.

18. Deployment & Rollback Strategy

Deployment Strategy

The Ticket Management System follows a controlled, version-based deployment process to ensure reliability, minimal downtime, and business continuity.

Step Activity

- 1 Promote changes through Development → Testing/UAT → Production environments.
- 2 Validate all features through functional testing, integration testing, and User Acceptance Testing (UAT).
- 3 Create a complete MongoDB Atlas backup before every production deployment.
- 4 Deploy approved version to the Hostinger VPS during scheduled low-impact maintenance windows.
- 5 Verify application health, APIs, dashboards, notifications, and reporting after deployment.
- 6 Obtain business validation and stakeholder approval before closing the deployment activity.

Production Deployment Checklist

- [] Latest source code pushed to Git repository
- [] Production branch/tag verified

- [] Environment variables (.env) updated and validated
- [] MongoDB Atlas backup completed successfully
- [] Backend application uploaded to Hostinger VPS
- [] Application dependencies installed (npm install)
- [] Frontend production build generated (npm run build)
- [] Nginx configuration verified (nginx -t)
- [] Nginx service reloaded
- [] Backend application restarted on Hostinger VPS
- [] HTTPS/SSL certificate verified
- [] Smoke Testing completed:
 - User Login
 - Ticket Creation
 - Ticket Assignment
 - Email Notifications
 - Dashboard Validation
 - Reporting
 - Work Hour Logging
- [] Stakeholders informed of successful deployment

Rollback Strategy

If any critical issue is identified after deployment, the system follows a structured rollback procedure to restore service as quickly as possible.

Step Rollback Action

- 1 Roll back the application to the last stable Git release/tag.
- 2 Restore MongoDB Atlas backup if database changes require reversal.
- 3 Redeploy the previous stable application version on the Hostinger VPS.
- 4 Validate authentication, ticket workflows, dashboards, reporting, and integrations.
- 5 Perform Root Cause Analysis (RCA), document findings, and schedule a corrected deployment.

Recovery Objectives

Metric	Target
Recovery Time Objective (RTO)	≤ 30 Minutes
Recovery Point Objective (RPO)	≤ 24 Hours (based on scheduled database backups)
Deployment Downtime Target	< 10 Minutes

Metric	Target
Rollback Validation	Authentication, Ticket Lifecycle, Reporting, Notifications, and Dashboard verification completed before production reopening.

19. Appendix — Glossary, Abbreviations & References

Glossary

Term	Definition
TMS	Ticket Management System — the enterprise platform used to manage customer and internal support requests, workflows, and service operations.
SLA	Service Level Agreement — the agreed response and resolution time targets for support tickets.
RBAC	Role-Based Access Control — a security model where user permissions are determined by assigned roles.
ERP	Enterprise Resource Planning — business management software integrating core organizational processes (e.g., SAP Business One, CREST ERP).
CRM	Customer Relationship Management — systems used to manage customer information, interactions, and sales processes.
HRMS	Human Resource Management System — software used to manage employee information, attendance, leave, and HR operations.
JWT	JSON Web Token — a secure, URL-safe token used for user authentication and API authorization.
ODM	Object Document Mapper — a library (such as Mongoose) that maps application objects to MongoDB documents.
API	Application Programming Interface — a set of interfaces enabling communication between software systems.
REST API	Representational State Transfer API — a web service architecture used for communication between client and server applications.
SMTP	Simple Mail Transfer Protocol — protocol used for sending automated email notifications.
CC	Carbon Copy — additional recipients included in system-generated email notifications.
AMC	Annual Maintenance Contract — an agreement covering ongoing support and maintenance services.
UAT	User Acceptance Testing — the validation phase where business users verify that the application meets operational requirements before production release.

RTO	Recovery Time Objective — the maximum acceptable time required to restore system operations after a failure.
RPO	Recovery Point Objective — the maximum acceptable amount of data loss measured by the time between backups.
KPI	Key Performance Indicator — measurable metrics used to evaluate operational and business performance.
AI	Artificial Intelligence — technologies used to automate decision-making, analyze data, and provide intelligent recommendations.

Abbreviations

Abbreviation Full Form

ASTPL	Akshay Software Technologies Private Limited
SAP	Systems, Applications, and Products in Data Processing
SAP B1	SAP Business One
HANA	High-Performance Analytic Appliance
CREST	Internal ERP platform supported by ASTPL
FP	Feature Pack
API	Application Programming Interface
RBAC	Role-Based Access Control
JWT	JSON Web Token
ODM	Object Document Mapper
SMTP	Simple Mail Transfer Protocol
UAT	User Acceptance Testing
RTO	Recovery Time Objective
RPO	Recovery Point Objective
KPI	Key Performance Indicator
AI	Artificial Intelligence
CI/CD	Continuous Integration / Continuous Deployment
VPS	Virtual Private Server

Abbreviation Full Form

UI	User Interface
UX	User Experience

References

Reference	Description
Internal IT Governance Guidelines	ASTPL internal standards governing software development, deployment, security, and operational governance.
Support Process & SLA Policy	Defines service levels, response targets, escalation procedures, and operational responsibilities.
Secure Application Development Standards	Internal coding standards, authentication policies, API security guidelines, and development best practices.
System Architecture Documentation	Technical architecture, database design, API specifications, deployment architecture, and workflow documentation for the Ticket Management System.
User & Administrator Guide	Functional documentation for Client Users, Consultants, Administrators, and Super Administrators.
Change Management & Release Documentation	Version history, release notes, deployment procedures, rollback strategy, and roadmap planning.
MongoDB Documentation	Official documentation for MongoDB database administration and development.
Node.js Documentation	Official Node.js runtime and API reference documentation.
Express.js Documentation	Official Express.js framework documentation.
React Documentation	Official React frontend framework documentation.
Mongoose Documentation	Official Mongoose ODM documentation for MongoDB.
Hostinger Documentation	VPS deployment, server administration, and hosting documentation.
Nginx Documentation	Official Nginx web server and reverse proxy documentation.

Appendix Summary

This appendix provides the terminology, abbreviations, and technical references used throughout the Ticket Management System (TMS) documentation. It serves as a quick reference for

developers, administrators, project stakeholders, auditors, and business users, ensuring consistent understanding of technical concepts, enterprise terminology, and implementation standards.

20. File Structure Overview

The Ticketing Management System follows a modular and scalable architecture to ensure maintainability and future extensibility.

Ticketing System version 5.0.0.0/

└─ client/	# React Frontend
└─ public/	# Static public assets
└─ src/	
└─ assets/	# Logos, images, static files
└─ components/	# Reusable UI component library
└─ ui/	# Core UI blocks (inputs, buttons, modals)
└─ layout/	# Navbar, Sidebar, Footer, MainLayout
└─ features/	# Role-specific feature panels
└─ user/	# Client user flows (dashboard, create ticket)
└─ consultant/	# Consultant workstation and dashboard
└─ superadmin/	# Admin console, routing rules, user management
└─ store/	# Zustand state containers (per domain)
└─ useAuthStore.js	# Authentication session state
└─ useTicketStore.js	# Ticket CRUD and lifecycle state
└─ useThemeStore.js	# Theme customization engine
└─ usePreAssignmentRuleStore.js	
└─ useClientStore.js	
└─ useDepartmentStore.js	
└─ useConsultantStore.js	
└─ App.jsx	# App root, router, theme provider
└─ index.css	# Global CSS variable system + overrides
└─ main.jsx	# React DOM mount point
└─ vite.config.js	# Vite build configuration
└─ package.json	

```
|
|
| └─ server/                # Node.js Express Backend
|   |
|   | └─ src/
|   |   |
|   |   | └─ config/
|   |   |   |
|   |   |   | └─ db.js      # MongoDB connection handler
|   |   |   |
|   |   |   | └─ controllers/    # Express route handlers
|   |   |   |   |
|   |   |   |   | └─ ticket.controller.js
|   |   |   |   |
|   |   |   |   | └─ auth.controller.js
|   |   |   |   |
|   |   |   |   | └─ preAssignmentRule.controller.js
|   |   |   |   |
|   |   |   |   | └─ consultantStats.controller.js
|   |   |   |   |
|   |   |   |   | └─ ccEmail.controller.js
|   |   |   |
|   |   |   | └─ middleware/
|   |   |   |   |
|   |   |   |   | └─ auth.middleware.js  # JWT protect + restrictTo guards
|   |   |   |   |
|   |   |   |   | └─ upload.middleware.js # Multer file parser configuration
|   |   |   |
|   |   |   | └─ models/
|   |   |   |   |
|   |   |   |   | └─ ClientUser.js      # All user types (clients, consultants, admins)
|   |   |   |   |
|   |   |   |   | └─ Client.js          # Client company + ERP details
|   |   |   |   |
|   |   |   |   | └─ Ticket.js          # Ticket lifecycle schema
|   |   |   |   |
|   |   |   |   | └─ Department.js      # Departments with categories
|   |   |   |   |
|   |   |   |   | └─ PreAssignmentRule.js # Routing rule configurations
|   |   |   |   |
|   |   |   |   | └─ CcEmailConfig.js   # CC email mapping configurations
|   |   |   |   |
|   |   |   |   | └─ Notification.js    # In-app notification records
|   |   |   |   |
|   |   |   |   | └─ TimeEntry.js       # Consultant work hour logs
|   |   |   |   |
|   |   |   |   | └─ ConsultantProfile.js # Extended consultant details
|   |   |   |
|   |   |   | └─ routes/                # Express route definitions
|   |   |   |   |
|   |   |   |   | └─ ticket.routes.js
|   |   |   |   |
|   |   |   |   | └─ auth.routes.js
|   |   |   |   |
|   |   |   |   | └─ preAssignmentRule.routes.js
|   |   |   |   |
|   |   |   |   | └─ ccEmail.routes.js
|   |   |   |
|   |   |   | └─ services/              # Business logic services
```

```
| | | |— ticket.service.js      # Ticket creation + routing evaluation
| | | |— consultantStats.service.js
| | | |— dashboard.service.js
| | | |— email.service.js
| | |— app.js                  # Express app initialization
| |— .env.example              # Environment variable template
|— package.json
|
└ docs/                        # System Documentation
  |— README.md                 # Documentation index and overview
  |— TMS_SYSTEM_DOCUMENTATION.md # This document
  |— USER_GUIDE_CLIENT.md      # Client user guide
  |— USER_GUIDE_CONSULTANT.md   # Consultant user guide
  |— USER_GUIDE_SUPERADMIN.md   # Super Admin guide
  └ TECHNICAL.md                # Technical architecture reference
...
---
```

Architecture Summary

Layer	Technology	Role
Presentation Layer	React 18 + Vite	Responsive role-based user interfaces, dashboards, ticket management, reporting, and administrative panels.
		Centralized client-side state management with domain-specific stores for authentication, tickets, users, notifications, and settings.
State Management	Zustand	
UI & Theming	Vanilla CSS + Tailwind CSS + CSS Custom Variables	Responsive design system with dynamic light, dark, and system theme support using CSS variables.

Layer	Technology	Role
API Layer	Node.js + Express.js	RESTful API development, request routing, middleware execution, authentication, validation, and business service orchestration.
Business Logic Layer	Service Layer (Node.js)	Implements ticket lifecycle management, automated routing, SLA monitoring, notifications, work-hour tracking, reporting, and business rules.
Authentication & Authorization	JWT + bcrypt	Secure user authentication, password encryption, Role-Based Access Control (RBAC), and protected API access.
Database Layer	MongoDB Atlas + Mongoose	Document-oriented NoSQL database with schema validation, ObjectId relationships, indexing, aggregation, and scalable data management.
File Management	Multer + MongoDB	Secure document upload, attachment management, and binary file storage associated with support tickets.
Email Service	Nodemailer (SMTP)	Automated email notifications, ticket updates, escalation alerts, CC email distribution, and workflow communications.
Hosting Platform	Hostinger VPS	Linux-based Virtual Private Server hosting the backend application, APIs, server-side services, and deployment environment.
Web Server & Reverse Proxy	Nginx	Serves the frontend application, handles HTTPS/SSL, routes API requests, and acts as a reverse proxy for backend services.
Version Control	Git	Source code management, version control, release tracking, and collaborative development.
Future AI Layer	AI Services (Planned)	AI-powered ticket classification, conversation understanding, automatic work-time calculation, intelligent routing, solution recommendations, and predictive analytics.

21. Conclusion & Management Summary

The Ticket Management System (TMS) is a modern, scalable, and enterprise-ready service management platform developed by Akshay Software Technologies Private Limited to streamline support operations, improve service quality, and establish a centralized, standardized approach to managing customer and internal service requests.

Since its initiation on 15 December 2025, the project has evolved through five major releases, culminating in the successful production deployment of Version v5.0.0.0. The platform now provides

a secure, configurable, and highly efficient solution for ticket lifecycle management, consultant workload tracking, SLA monitoring, reporting, and business workflow automation.

The system delivers measurable business value across four key dimensions:

Operational Excellence

- Centralized and standardized ticket management replacing fragmented manual processes.
- Automated ticket routing, configurable workflows, and streamlined support operations.
- Integrated work-hour tracking, consultant management, and real-time operational dashboards.
- Reduced response times and improved service delivery through automation.

Governance & Compliance

- Enterprise-grade Role-Based Access Control (RBAC) with secure JWT authentication.
- Comprehensive audit trails, assignment history, activity logs, and timestamped records.
- SLA monitoring, escalation workflows, and management reporting for operational governance.
- Secure deployment with structured backup, recovery, and version-controlled release management.

Business Intelligence & Decision Support

- Interactive dashboards with consultant performance metrics, workload analysis, client-wise reporting, and KPI monitoring.
- Automated reporting with advanced filtering and Excel export capabilities.
- Real-time visibility into operational performance, resource utilization, and service quality.

Strategic Growth & Innovation

- Scalable architecture supporting multi-company operations and enterprise expansion.
- Designed for seamless integration with ERP, CRM, HRMS, Email, WhatsApp, Microsoft Teams, and third-party applications.
- AI-ready platform with a roadmap that includes intelligent ticket classification, conversation understanding, automatic work-time calculation, AI-powered solution recommendations, predictive analytics, consultant recommendation engines, and executive business intelligence.

Management Summary

The Ticket Management System (TMS) is fully operational and production-ready, providing a stable foundation for enterprise service management while supporting continuous business growth and digital transformation.

By combining modern web technologies, enterprise security, workflow automation, comprehensive reporting, and a future-focused AI strategy, the platform is positioned to become a strategic Enterprise Service Management (ESM) solution that delivers long-term value to management, support teams, consultants, and client organizations.

The project demonstrates a scalable architecture, maintainable codebase, and structured release methodology, ensuring the platform can continuously evolve to meet changing business requirements and future technological advancements.

Final Statement

Version v5.0.0.0 represents the successful completion of the initial enterprise implementation of the Ticket Management System. Future releases will focus on intelligent automation, AI integration, enterprise analytics, and expanded business process integration, reinforcing the platform's role as a core enterprise solution within Akshay Software Technologies Private Limited.

Document Information	Details
Document Name	Ticket Management System (TMS) – Technical Design, Architecture & User Documentation
Document Version	v5.0.0.0
Organization	Akshay Software Technologies Private Limited
Project Duration	15 December 2025 – 03 July 2026
Prepared By	Ruhul Amin (Lead Full Stack Developer)
Reviewed By	Rajan Chelladurai, Reshma Lokhande
Document Status	Approved for Production Release
Classification	Internal – Confidential
Next Planned Release	v5.1.0.0 – AI & Enterprise Automation Enhancements

Final Approval & Signatures

Document Approval

This document certifies that the **Ticket Management System (TMS) Version v5.0.0.0** has been reviewed, validated, and approved for production deployment and operational use within **Akshay Software Technologies Private Limited**.

The undersigned acknowledge that the system has been developed in accordance with the approved business requirements, technical specifications, security standards, and quality expectations. Any

future modifications or enhancements will be governed through the organization's change management and version control processes.

Project Approval Matrix

Role	Name	Signature	Date
Project Sponsor / Product Owner	Akshay Software Technologies Private Limited		
Operations Head	Rajan Chelladurai		
Technical Head / Engineering Manager	Ruhul Amin		
Quality Assurance (QA)	Reshma Lokhande		
Lead Full Stack Developer	Ruhul Amin		

Management Acceptance

We hereby confirm that the **Ticket Management System (TMS)** has successfully completed the required development, testing, validation, and review processes. Based on the documented evidence and system evaluation, the solution is approved for production use and ongoing operational support.

This approval confirms that:

- The implemented functionality satisfies the approved business requirements.
- Security, authentication, and authorization mechanisms have been validated.
- The system architecture supports future scalability and enterprise integration.
- Backup, recovery, deployment, and rollback procedures have been documented.
- The platform is ready for continuous enhancement through future version-controlled releases.

Developer Declaration

I hereby declare that the **Ticket Management System (TMS) Version v5.0.0.0** has been designed, developed, tested, documented, and deployed following industry best practices and the technical requirements defined by **Akshay Software Technologies Private Limited**.

All source code, database structures, APIs, workflows, documentation, deployment procedures, and release artifacts have been prepared to support maintainability, scalability, and future enhancements.

Developer Name: Ruhul Amin
Designation: Lead Full Stack Developer

Signature: _____

Date: _____

Project Sponsor Declaration

I confirm that the Ticket Management System has been reviewed against the approved business objectives and is accepted for production deployment and operational use within the organization.

The system provides a scalable foundation for future enterprise enhancements, automation initiatives, and AI-driven capabilities.

Name: Rajan Chelladurai

Designation: Operations Head

Signature: _____

Date: _____

END OF DOCUMENT

Ticket Management System (TMS)

Version: v5.0.0.0

Prepared By: Ruhul Amin

Organization: Akshay Software Technologies Private Limited

Document Classification: Internal – Confidential

© 2026 Akshay Software Technologies Private Limited. All Rights Reserved.